HUST

ĐẠI HỌC BÁCH KHOA HÀ NỘI

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.

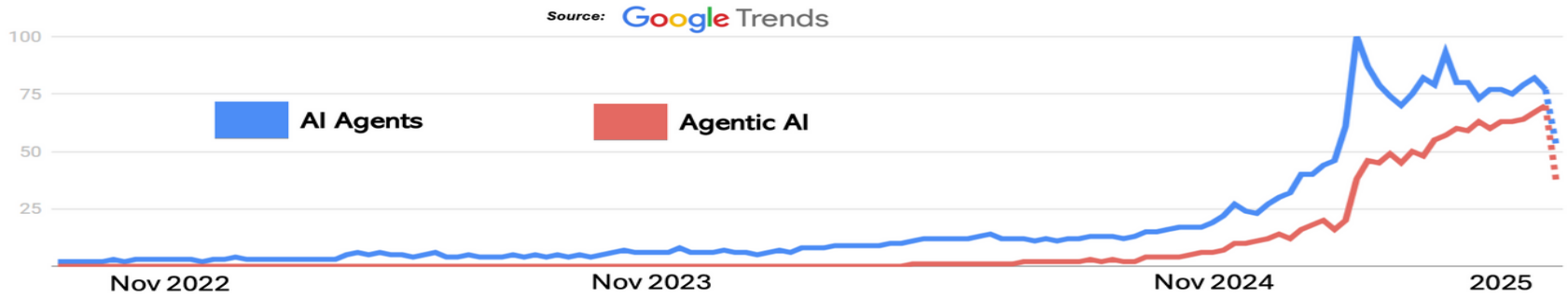
ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Agentic AI

Ngô Văn Linh

ONE LOVE. ONE FUTURE.

Trending in AI Agents and Agentic AI



Global Google search trends showing rising interest in “AI Agents” and “Agentic AI” since November 2022 when the ChatGPT was first introduced. [1]

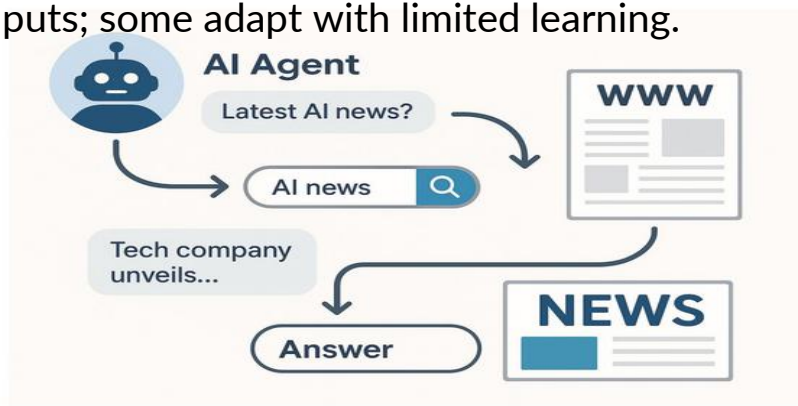
Outline

1. Introduction to Agentic AI
2. MCP (Model Context Protocol)
3. A2A (Agent to Agent)
4. Exercises

1.1 AI Agents

- **Definition**: Autonomous software entities that perform tasks based on predefined goals. They perceive inputs, reason, and act to achieve objectives, often without human intervention.
- **Core Characteristics:**
  - Autonomy: Operate independently, reacting to dynamic inputs.
  - Task-Specificity: Built for specific, well-defined tasks.
  - Reactivity & Adaptation: Respond to real-time inputs; some adapt with limited learning.

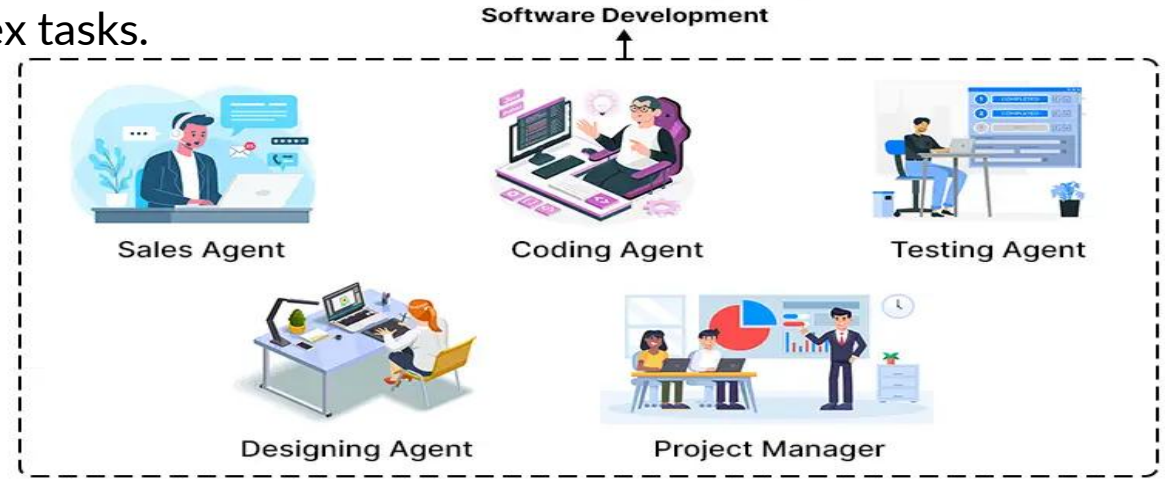
Agent Characteristics [1]



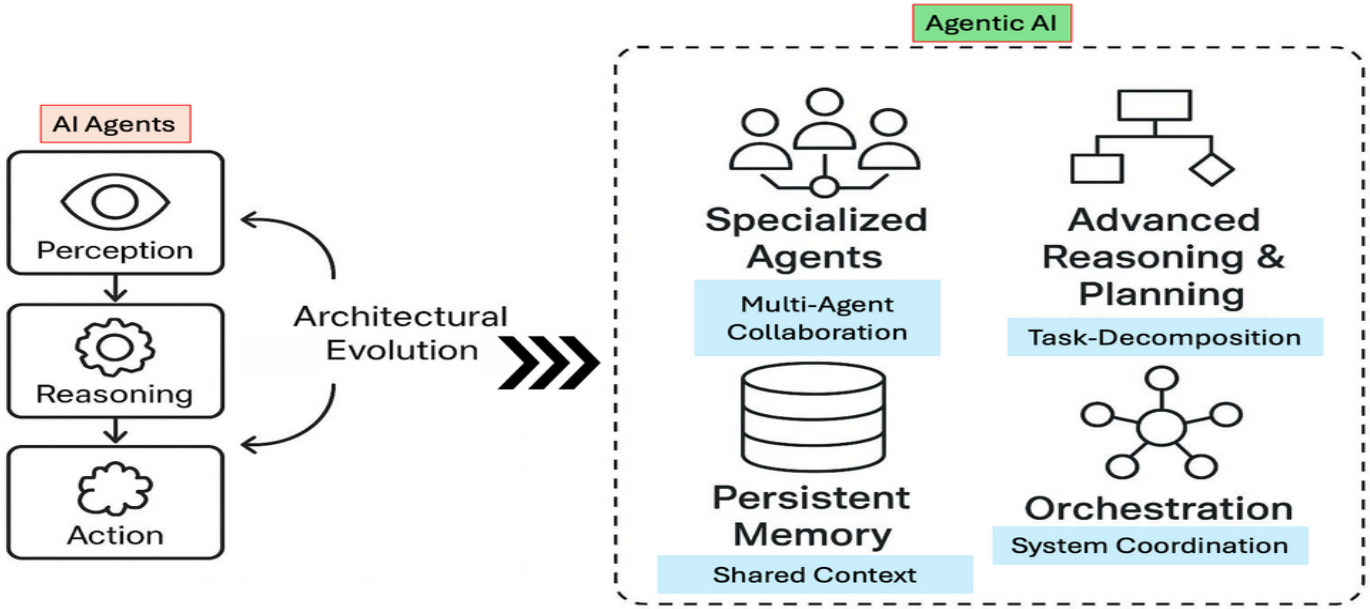
Workflow of a news research agent [1]

1.2 Agentic AI

- **Definition:** A collaborative system of AI agents that coordinate to achieve complex, high-level goals with shared objectives, distributed tasks, and enhanced autonomy.
- **Key Difference from AI Agents:**
  - **AI Agents:** Standalone, task-specific, autonomous entities.
  - **Agentic AI:** Multiple agents dynamically collaborating, adapting in real-time to tackle complex tasks.

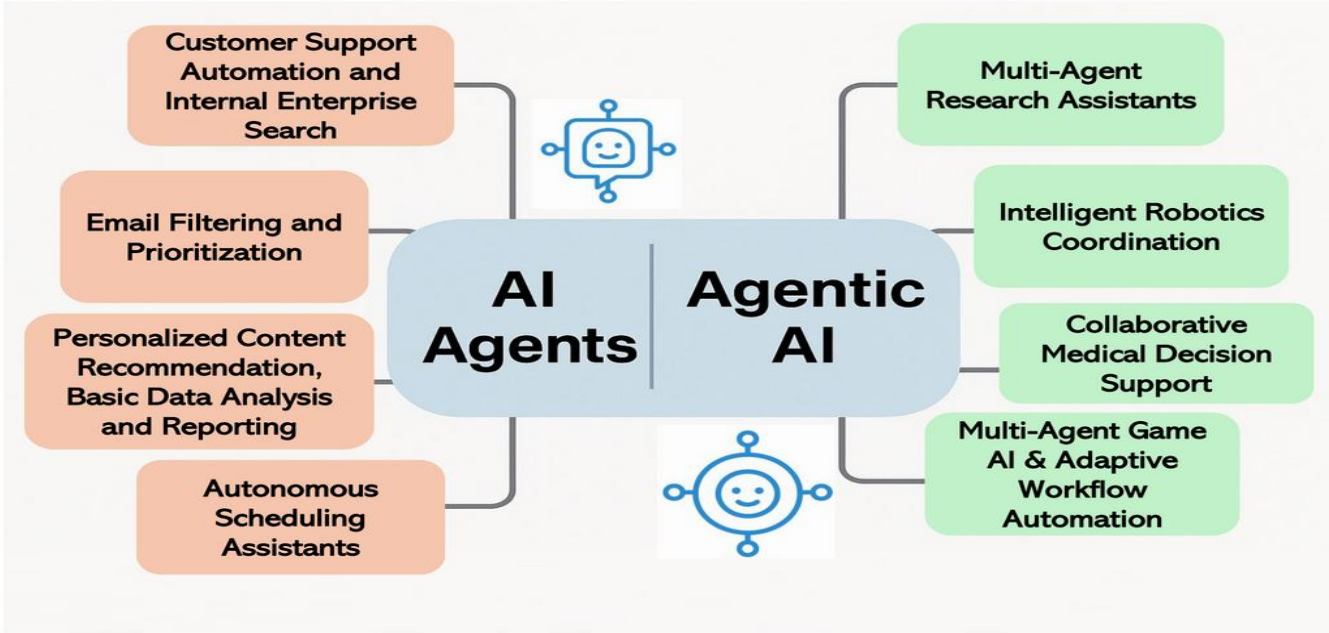


1.3 Agentic AI vs AI Agents



Difference by architecture [1]

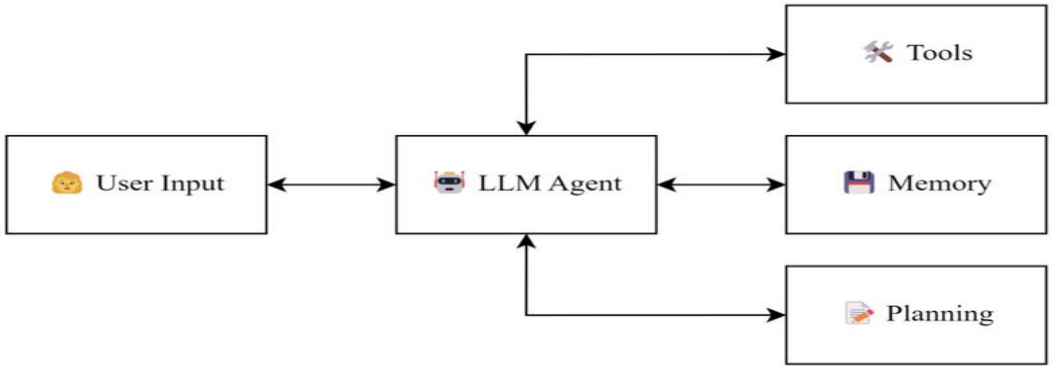
1.3 Agentic AI vs AI Agents (cont.)



Difference by application [1]

1.4 How LLMs Power AI Agents

- **LLMs are Core Language Understanding Engines for AI Agents**
  - Interprets user inputs (Language Comprehension)
  - Produces natural responses (Text Generation)
  - Supports language-related tasks (Task Support, Tool calling)



AI Agent with LLM [2]

1.4 Problems in Agentic AI

- ❌ No standard way for agents to use tools or talk to each other
- 🔒 Privacy & security risks when tools are tightly coupled to LLMs
- 🧱 Hard to scale: each agent = custom logic, no reuse
- 🤝 Multi-agent collaboration is still manual and fragmented

- 👉 We need standard rules for agent communication & tool use

1.5 Why Agent Protocols?

- Agent protocols solve:
  - ✅ Tool access: One standard way to call tools
  - ✅ Agent collaboration: Agents can talk and work together
  - ✅ Security: Sensitive data handled safely
  - ✅ Scalability: Plug-and-play new tools/agents

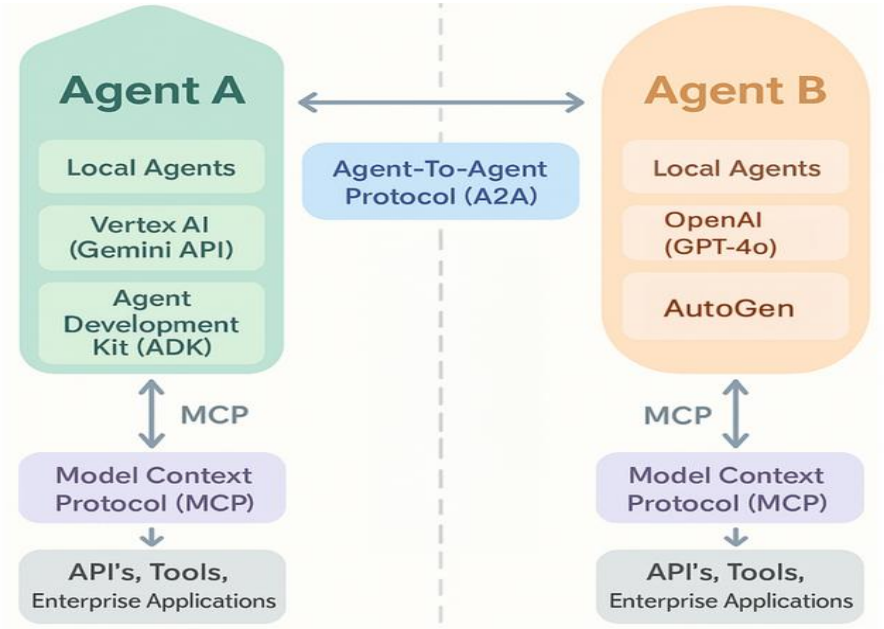
- 💡 Standardization → unlocks intelligent networks

1.6 Trending Agent Protocols

- 🧠 MCP (Model Context Protocol – Anthropic)
  - Agents use tools via a clean interface
  - Safer: LLM ≠ execute tools
  - Scalable, privacy-first
- 👉 MCP = Agent ↔ Tool

- 🤝 A2A (Agent-to-Agent – Google)
  - Agents discover, talk, share tasks
  - Async-ready, secure, multi-modal
  - Built for enterprise collaboration
- 👉 A2A = Agent ↔ Agent

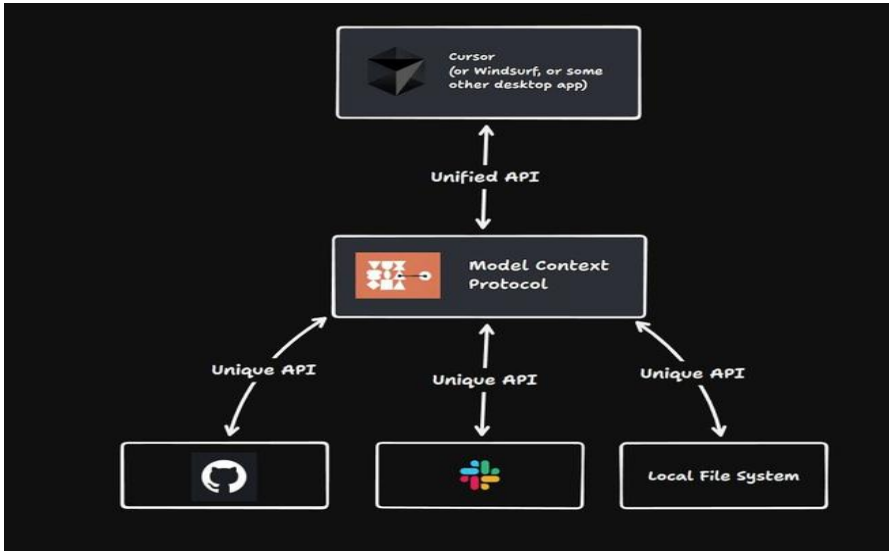
1.6 Trending Agent Protocols



Multi Agents workflow with MCP and A2A

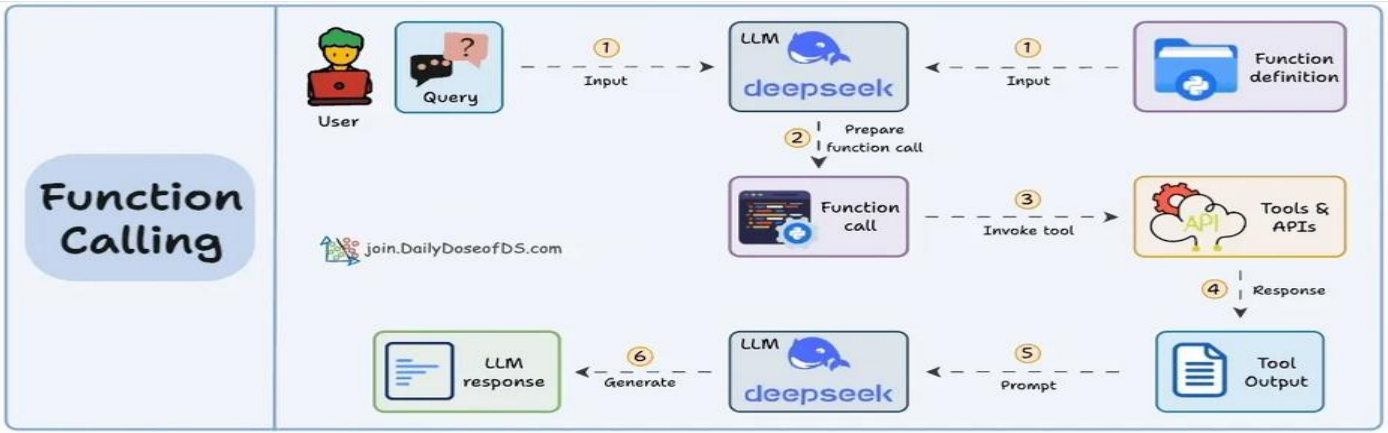
2. Model context protocol (MCP)

- **What is MCP?**
  - An open standard introduced by Anthropic for integrating external data, tools, and context into AI models.
- Acts as a “universal port” to standardize interactions between AI agents and external systems (APIs, databases, tools).



2. MCP vs Function Calling

- What is Function Calling?
  - Function Calling is a mechanism that allows an LLM to recognize what tool it needs based on the user's input and when to invoke it.
- Here's how it typically works:
  - The LLM receives a prompt from the user.
  - The LLM decides the tool it needs.
  - The programmer implements procedures to accept a tool call request from the LLM
  - The : would han



2. MCP vs Function Calling

```
<|system|>
You are a helpful assistant. You can call tools when needed to answer the user's questions.

<|tools|>
[
  {
    "name": "get_current_weather",
    "description": "Get the current weather for a given location",
    "parameters": {
      "type": "object",
      "properties": {
        "location": {
          "type": "string",
          "description": "The city to get the weather for"
        }
      },
      "required": ["location"]
    }
  }
]

<|user|>
What's the weather in Tokyo today?

<|tool_call|>
{"name": "get_current_weather", "arguments": {"location": "Tokyo"}}
```

2. MCP vs Function Calling

```python
# Our mock tool
def get_current_weather(location: str) -> dict:
    # In real use, this would be an API call
    return {
        "location": location,
        "temperature": "28°C",
        "condition": "Partly cloudy"
    }
```

```python
def handle_tool_call(llm_output):
    if "tool_call" in llm_output:
        tool_name = llm_output["tool_call"]["name"]
        arguments = llm_output["tool_call"]["arguments"]

        # You can add multiple tools here
        if tool_name == "get_current_weather":
            result = get_current_weather(**arguments)
            return {
                "tool_response": result
            }
        else:
            raise ValueError(f"Unknown tool: {tool_name}")
    else:
        # No tool call needed
        return {"tool_response": None}
```

```
<|tool_response|>
{"location": "Tokyo", "temperature": "28°C", "condition": "Partly cloudy"}

<|assistant|>
The current weather in Tokyo is 28°C with partly cloudy skies.
```

2. Custom Integrations vs MCP

Custom Integrations FC

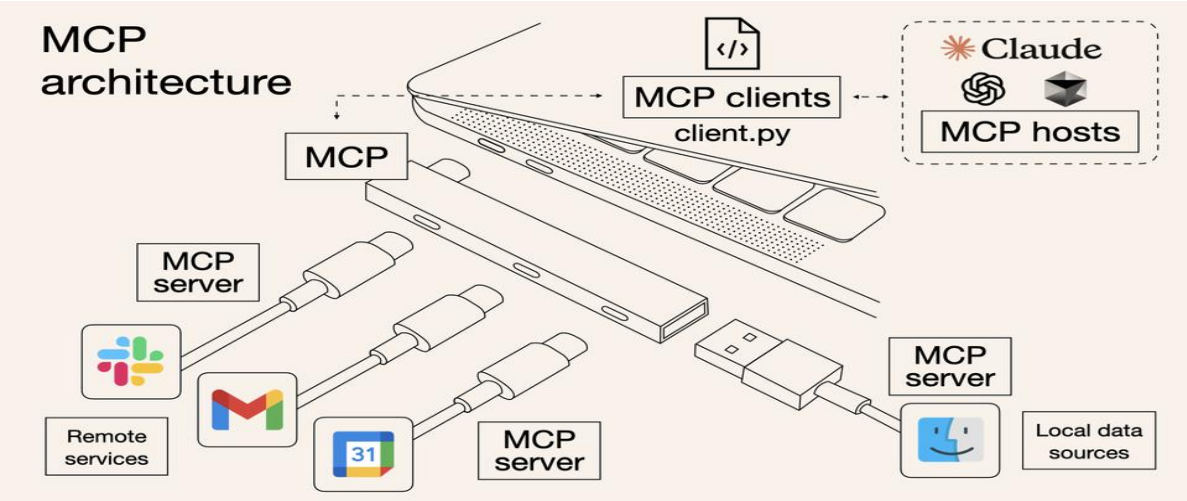
- High development time
- Low scalability
- High maintenance
- High flexibility (customized)

MCP

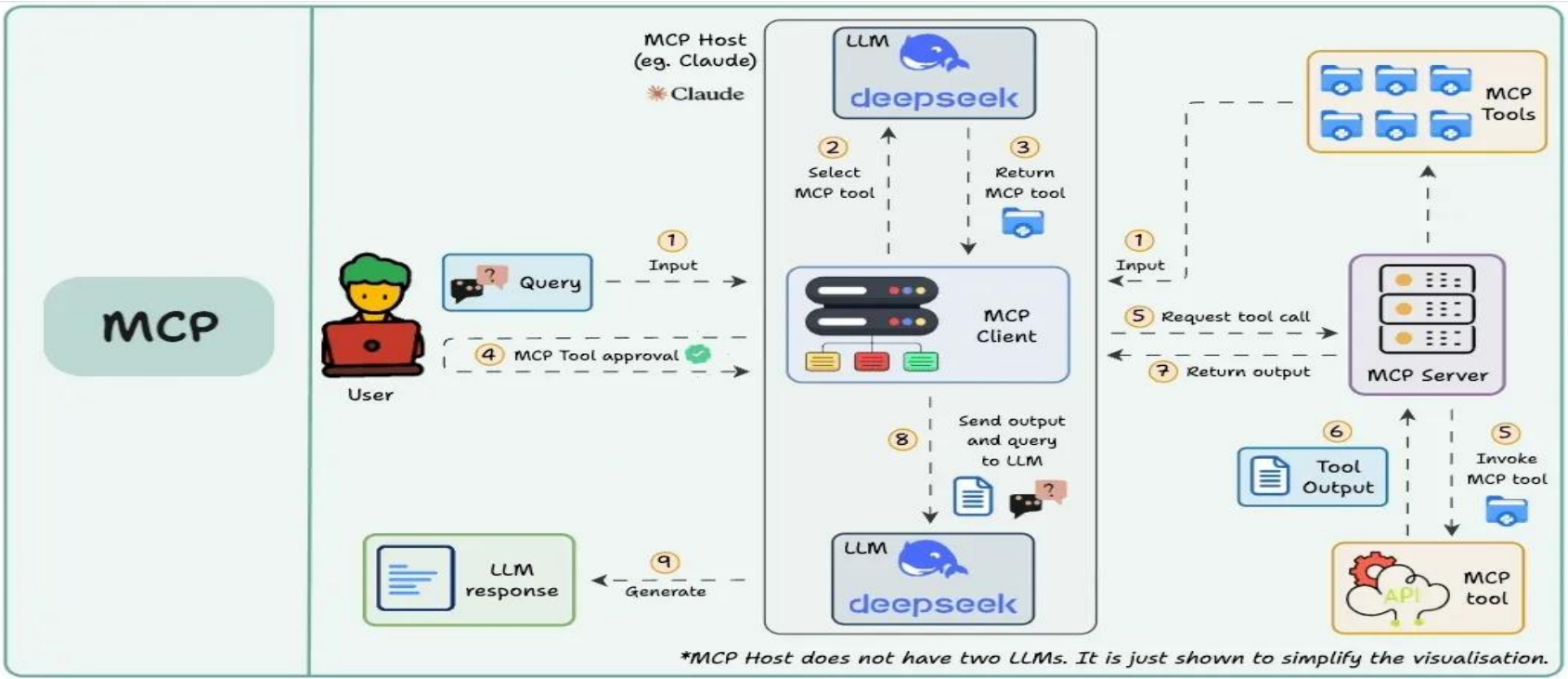
- Low development time
- High scalability
- Low maintenance
- Standardized

2.1 MCP Architecture

- **Host**: Main AI application the user interacts with (e.g., Claude Desktop, Cursor IDE, ...).
- **Client**: Manages connection to one MCP Server, sandboxes interactions.
- **Server**: Exposes tools/resources/prompts related to external systems.

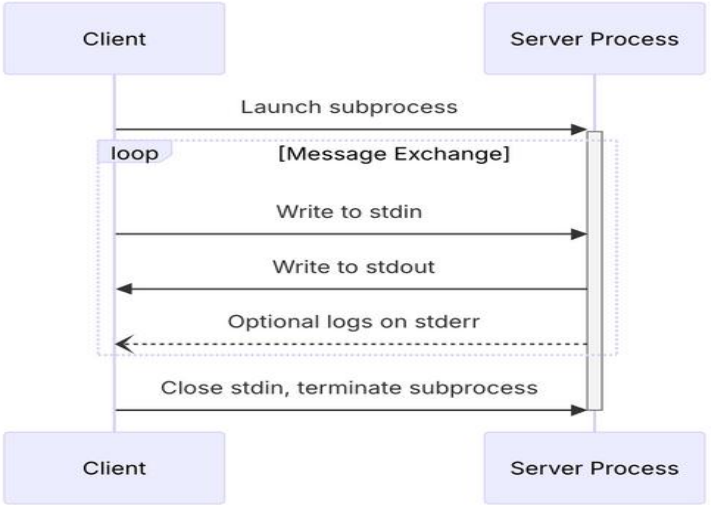


2.1 MCP example Workflow



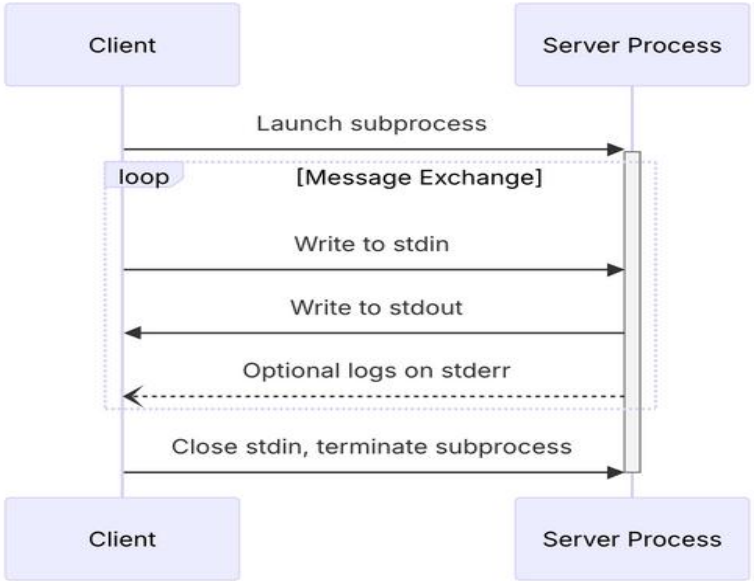
2.1 Transport & Communication in MCP: stdio

- **What it is**:
  - A **local** communication method where the **client** launches the MCP **server** as a **subprocess**.
  - The server reads messages from standard input (stdin) and writes responses to standard output (stdout).
  - Messages exchanged are encoded in JSON-RPC, following the protocol's structure.



2.1 Transport & Communication in MCP: stdio (cont)

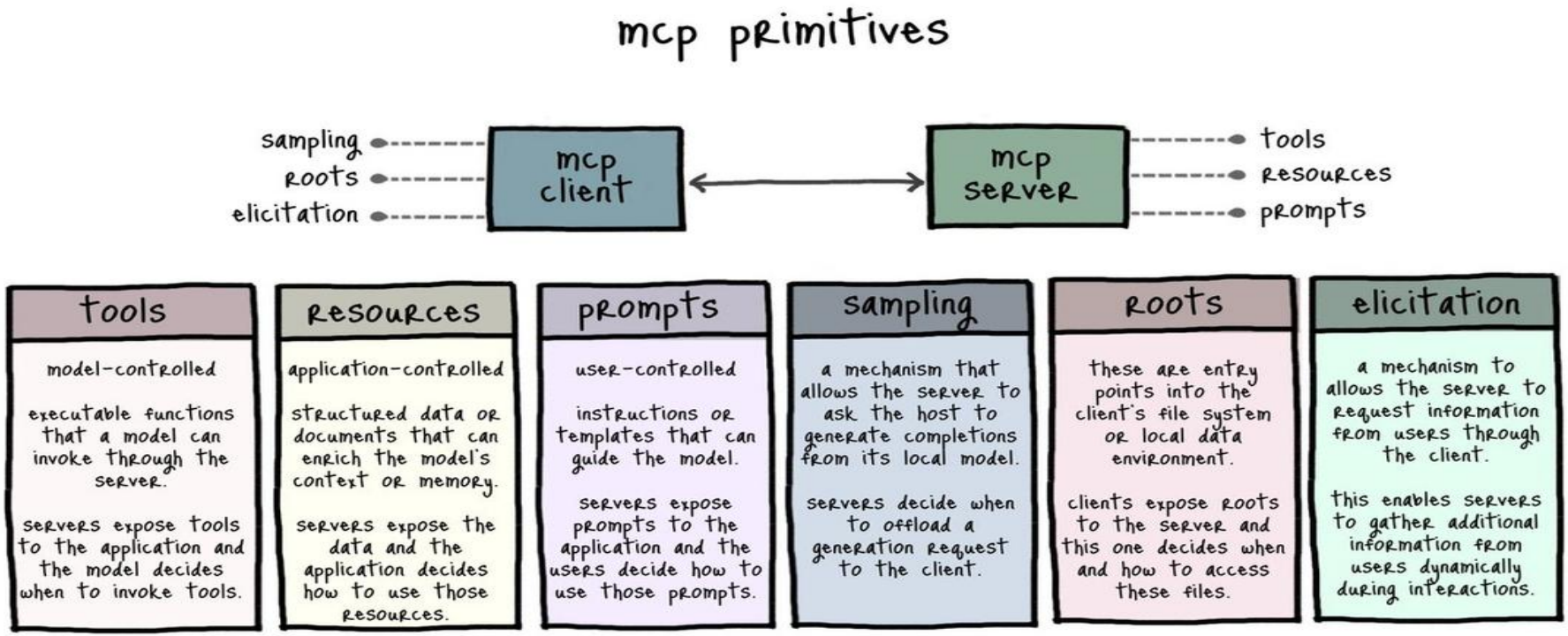
- **Advantages**:
  - Simple and efficient for local communication.
  - Low latency and fast message exchange.
- **Important Notes:**
  - Messages must be valid JSON-RPC and must not contain embedded newlines.
  - The server may write logs to stderr but should not send anything other than valid MCP messages through stdout.



2.1 Transport & Communication in MCP: streamable-http

- **HTTP-based** transport between client and server
- Client → Server: HTTP POST carrying JSON-RPC messages to the MCP endpoint
- Server → Client: streams responses via Server-Sent Events (SSE)
- Server runs independently → handles multiple clients at once
- Enables real-time, asynchronous communication
- Uses POST (requests) + GET (receive streamed responses)

2.1 MCP Primitives



2.1 MCP Resources

- Core primitive for exposing server data to LLM clients Application-controlled: Clients decide how/when to use them
- Examples of resources:
  - File contents (e.g., source code, logs)
  - Database records
  - API responses
  - Screenshots/images
- Identified by unique URIs

2.1 Types of Resources

Text Resources

- UTF-8 encoded text
- Suitable for:
  - Source code
  - Logs
  - JSON/XML

Binary Resources

- Base64-encoded data
- Suitable for:
  - Images PDFs
  - Audio/Video

2.1 MCP Prompts

Definition

A **Prompt** in the Model Context Protocol (MCP) is a reusable, structured input that guides how an LLM should respond or behave.

- Think of it as a "program" or "template" for LLMs.
- Includes the task, structure, and sometimes arguments.
- Not just text — it's metadata + logic.

2.1 MCP Prompts

Why Prompts Matter

- Prompts **control behavior** of LLMs — they define the task.
- Enable **repeatability**: same prompt → consistent behavior.
- Help abstract complex workflows into simple, user-friendly commands.
- Allow server-defined logic that users can interact with safely.

Essence

Prompts = **Instruction + Context + Argument Inputs**

What Are MCP Tools?

- **MCP Tools** allow servers to expose executable operations (functions) to be used by LLMs.
- Tools can perform actions like querying APIs, modifying databases, or executing commands.
- They are an abstraction for **external function calling**.
- **LLMs can discover and invoke tools** as if calling structured functions.
- Think of tools as an LLM's way to "reach out" to the world beyond text.

Analogy

Tools = APIs for LLMs to interact with the real world

Best Practices for Tool Design

Design Principles

- Clear name and description
- Input schema with types
- Error handling and validation
- Small, atomic logic
- Annotate readOnly/destructive

Why it matters for LLMs

- Helps model reason about effects
- Encourages safe execution
- Enables structured error reporting
- Boosts user trust + transparency

Deep Insight

LLMs don't just respond to text — they can **think, decide, and act** by calling tools. MCP enables this by turning language into executable logic.

2.1 MCP Sampling

- Sampling lets server request LLM through client
- Enable agentic workflow while preserving privacy and security
- Enable Human in the loop

2.1 MCP Sampling

MCP Sampling Flow

1. Server asks the Host to generate with an LLM
2. Host reviews the request for safety/context
3. Host selects a model and calls the LLM
4. Host checks the generated result
5. Result is returned back to the Server

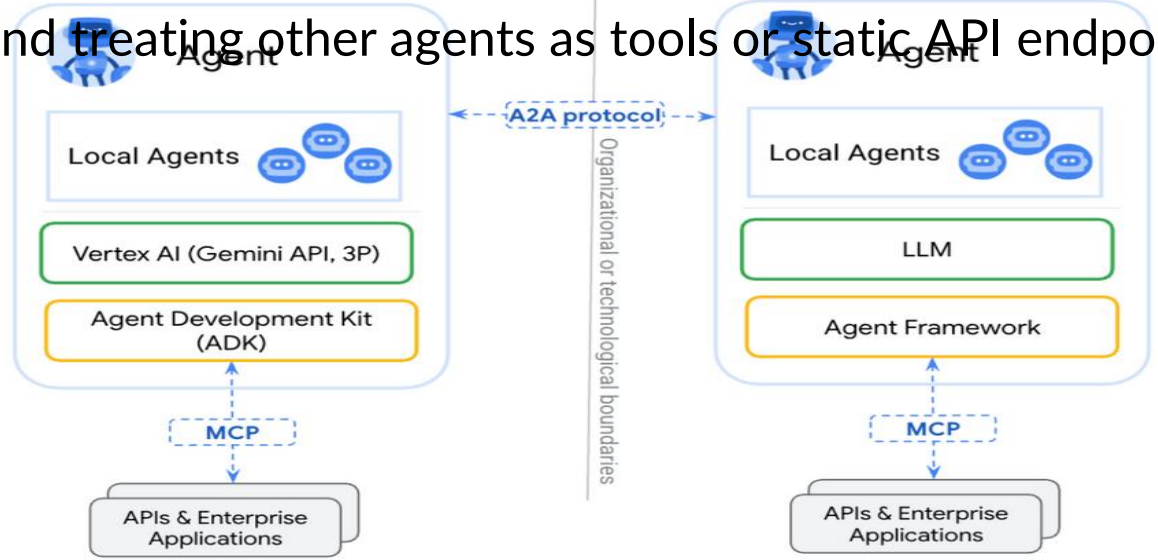
2.1 MCP Roots

- What are Roots?
  - A root is a URI (like a file path or URL) that a client suggests the server should focus on. It defines the **workspace** boundary.

```
{
  "roots": [
    { "uri": "file:///home/user/project", "name": "Project" },
    { "uri": "https://api.example.com/v1", "name": "API" }
  ]
}
```

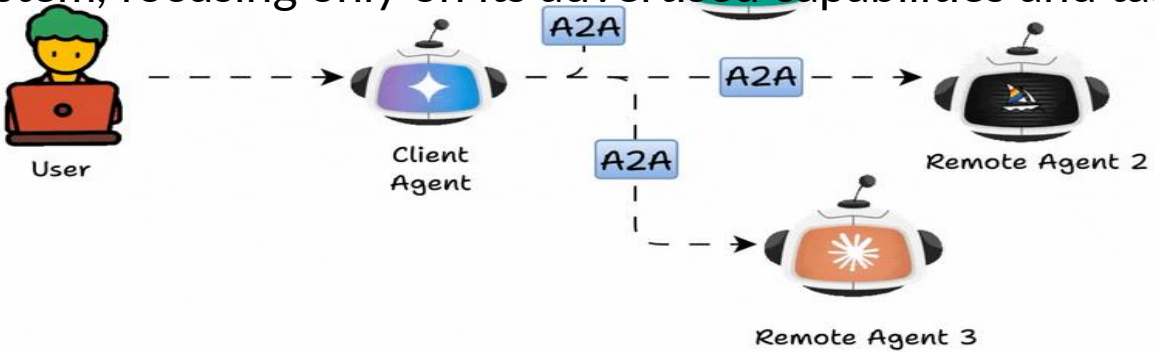
3. Agent to Agent (A2A)

- **Definition**: A2A is an open standard introduced by Google to enable direct communication and interoperability between AI agents across frameworks and vendors.
- **Key Goal:** Allow agents to collaborate and share information in a structured way, moving beyond treating other agents as tools or static API endpoints.



3.1 Core Actors in A2A

- User:
  - The human or automated service that initiates the request.
- A2A Client (Client Agent):
  - An application, service, or another AI agent acts on behalf of the user, initiating requests and interacting with the remote agent.
- A2A Server (Remote Agent):
  - Exposes an HTTP endpoint and processes tasks. The client interacts with it as an opaque system, focusing only on its advertised capabilities and tasks.



3.2 Fundamental Communication Elements

A2A Communication Objects

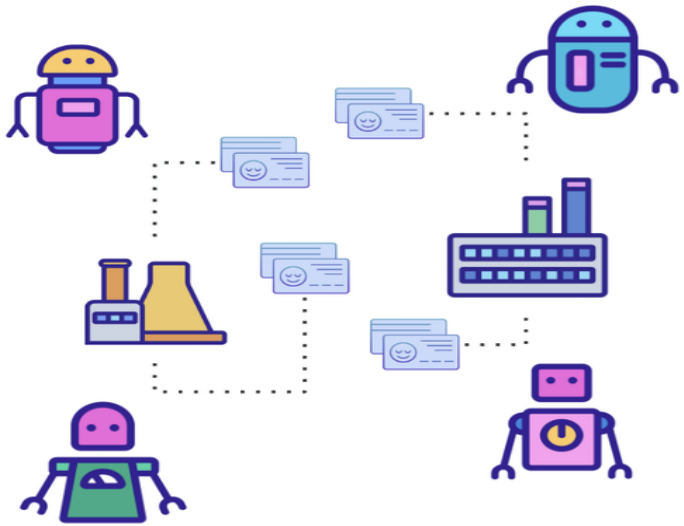
- Agent Card → Discover agents
- Task → Define the work
- Message → Exchange conversation/context
- Artifact → Return final outputs

From discovery to task execution, these objects define how agents communicate.

3.2 Agent Card

Google Cloud

A2A – Agent Card



Name: Agent ABC
Provider: Google (URL)
Preferred input/output: text

Capabilities:
I can do streaming
I can do Push Notifications

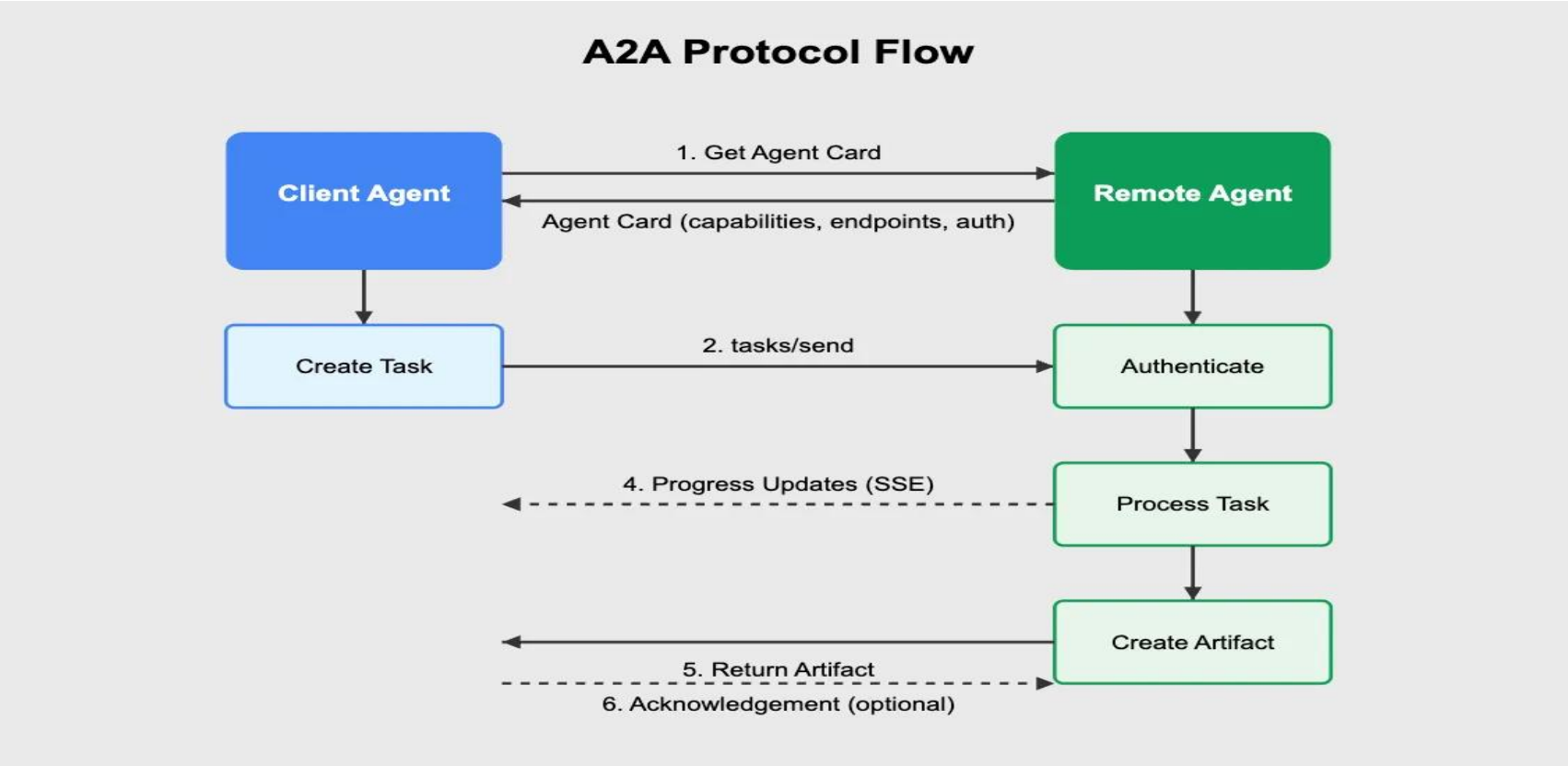
Skills:
- Skill A
  - Description
  - Input
  - Output
- Skill B
  - Description
  - Input
  - Output

Authentication:
Scheme: Bearer
Key: xxx

3.2 Core communication objects

- At the heart of A2A communication is the **Task**, i.e., a structured object that represents an atomic unit of work. **Tasks** have lifecycle **states** such as **submitted**, **working**, **input-required**, or **completed**.
- Each Task includes:
  - **Messages**: These are used for **conversational** exchanges between the Client and the Remote agent
  - **Artifacts**: These are for immutable **results** created **by** the **remote agent.**
  - **Parts**: Self-contained **data** blocks **within messages** or **artifacts** like plain text, file blobs, or JSON.

3.3 A2A Flow


A2A Protocol Flow
Client Agent
Remote Agent
1. Get Agent Card
Agent Card (capabilities, endpoints, auth)
Create Task
2. tasks/send
Authenticate
4. Progress Updates (SSE)
Process Task
Create Artifact
5. Return Artifact
6. Acknowledgement (optional)

3.4 A2A and MCP Pattern

- MCP: Agent connects to tools and data
- A2A: Agent collaborates with other agents
- MCP = vertical capability
- A2A = horizontal collaboration
- Together, they form a complete Agentic AI system

References

[1]https://arxiv.org/pdf/2505.10468
[2] https://arxiv.org/pdf/2503.23037
[3]https://arxiv.org/pdf/2401.03568